

Noun–Verb Analysis

Step 2: Noun-Verb Analysis (Simple & Clear Guide)

Project Overview

Course: B.Tech. CSE (Semester 5)

Subject: System Design (TCS-504)

Project: Movie Ticket Booking System

1. Noun Analysis Table (Deciding Classes)

Rule for Object-Oriented Design

- A noun becomes a **Class** if it has its own identity, holds data (state), and has behavior (methods).
- If a noun is just a single value (like a number or string) or a visual printout on the screen, we **reject** it as a class and make it an attribute or a method instead.

1.1 Accepted Nouns (Classes to Keep)

Noun Identified	Keep as Class?	Category / Role	What Data It Holds (Attributes)	Why We Keep It (Design Reason)
Movie	YES	Core Entity	title, language, durationInMinutes	Represents an independent movie with its own data.
Seat	YES	Core Entity	seatNumber (e.g. A1), seatType	Represents a physical chair in a cinema hall.
Screen	YES	Core Entity	screenId, screenName, list of Seats	Represents a physical hall that holds seats.
Cinema	YES	Core Entity	cinemaName, list of Screens, list of Movies	The main container that manages screens and the movie catalog.
Show	YES	Core Entity	showId, Movie, Screen, startTime, list	A scheduled movie screening at a

Noun Identified	Keep as Class?	Category / Role	What Data It Holds (Attributes)	Why We Keep It (Design Reason)
			of ShowSeats	specific time.
ShowSeat	YES	Core Entity	Seat, status (AVAILABLE or BOOKED)	Tracks the booking status of a seat for ONE specific show.
Customer	YES	Core Entity	customerName, phoneNumber	Stores the contact information of the person booking.
Booking	YES	Core Entity	bookingId, Show, list of ShowSeats, totalAmount, status	Stores a confirmed booking transaction record.
Payment	YES	Abstract Base	None (Base Interface)	Defines the standard payment method pay(amount) .
UpiPayment	YES	Payment Type	upId	Class handling UPI payment steps.
CardPayment	YES	Payment Type	cardNumber, expiryDate	Class handling Card payment steps.
CashPayment	YES	Payment Type	None	Class handling Cash payment steps.
PriceCalculator	YES	Helper	Pricing rates for each seat type	Dedicated calculation logic to turn selected seats into a total bill.
TicketPrinter	YES	Helper	Ticket layout template	Dedicated service to format and print receipts on the screen.
BookingService	YES	Manager	List of all bookings, reference to cinema	The central manager that runs the entire booking process.

1.2 Rejected Nouns (What We Did NOT Make a Class)

Why reject nouns?

Rejecting a noun is also good design. Not every noun needs to be a class. Over-engineering makes code complicated.

Noun Identified	Keep as Class?	What It Actually Is	Why Rejected as a Class?
"Seat Layout"	NO	A Method	It is just a visual printout of a Show's seats. It is implemented as a method <code>displaySeatLayout()</code> .
"Price / Amount"	NO	A Simple Attribute	It is just a numeric value (<code>double</code>), computed dynamically by <code>PriceCalculator</code> .
"Start Time"	NO	A Simple Attribute	It is just a text value stored inside <code>Show</code> .
"Booking ID"	NO	A Simple Attribute	It is just a unique identifier string stored inside <code>Booking</code> .
"Menu / Program"	NO	A Main Loop	It is just the <code>main()</code> loop that asks for input, not a core business object.

2. Verb Analysis Table (Deciding Methods)

Rule for Methods

- Verbs in the problem statement represent **actions**.
- Always assign the method to the class that **holds the data needed** to perform that action.

Action (Verb)	Responsible Class	Method Name	Reason (Why This Class Owns It)
List Movies	Cinema	<code>getMovies()</code>	Cinema stores and manages the collection of all active movies.
List Shows	Cinema	<code>getShowsForMovie(movieId)</code>	Cinema holds the scheduled shows and knows which show belongs to which movie.
Display Seat Layout	Show	<code>displaySeatLayout()</code>	Show owns all the <code>ShowSeat</code> objects for that timing and can print them.
Check Seat	ShowSeat	<code>isAvailable()</code>	ShowSeat holds the AVAILABLE/BOOKED

Action (Verb)	Responsible Class	Method Name	Reason (Why This Class Owns It)
Availability			status for that specific seat.
Book / Lock Seat	ShowSeat	bookSeat()	ShowSeat is responsible for changing its own state to BOOKED.
Release / Free Seat	ShowSeat	releaseSeat()	ShowSeat is responsible for resetting its state back to AVAILABLE.
Calculate Total Price	PriceCalculator	calculateTotalPrice(seats)	PriceCalculator holds the pricing rules (Silver Rs. 150, etc.).
Process Payment	Payment (Child)	pay(amount)	Handled by specific payment classes like UpiPayment or CardPayment .
Print Ticket Receipt	TicketPrinter	printTicket(booking)	Dedicated utility class to format borders and text on the screen.
Handle Booking	BookingService	bookTickets(...)	Central manager that validates seats, calculates price, pays, and creates a Booking .
Cancel Booking	BookingService	cancelBooking(bookingId)	BookingService looks up the booking, cancels it, and frees the seats.
Lookup Booking	BookingService	findBooking(bookingId)	BookingService owns the list of all confirmed and cancelled bookings.

3. Common Design Mistakes (Explained Simply)

Guide for Beginners

Here is a simple guide on common mistakes beginners make, showing what not to do and how to fix it.

Mistake 1: Storing "isBooked" inside the physical Seat class

- **What NOT To Do:** Do not put a booking status inside the `Seat` class.
- **Real-Life Example:** A physical chair in a cinema is not booked forever. It is only booked for a specific show (like the 6:00 PM show). At 9:00 PM, someone else can sit on it.
- **Why it Fails:** If you put it in `Seat`, booking a seat at 6:00 PM will also block it for tomorrow's show.
- **What TO Do Instead:** Keep `Seat` independent. Create a separate class called `ShowSeat` to store the booking status for each specific show.

The Rule

Physical things (chairs) don't store time-based data (bookings).

Mistake 2: Making the Booking class print the ticket

- **What NOT To Do:** Do not write printing code inside the `Booking` class.
- **Real-Life Example:** An invoice in an accounting book is just data. It doesn't care if you print it on paper or email it.
- **Why it Fails:** If you want to change the ticket border style later, you would be forced to change your core database class.
- **What TO Do Instead:** Let `Booking` only hold data. Create a dedicated `TicketPrinter` class whose only job is formatting and printing.

The Rule

A data class should only store data. Keep printing in a separate Printer class.

Mistake 3: Writing long IF-ELSE for payment types

- **What NOT To Do:** Do not write long if-else blocks for UPI, Card, and Cash inside the booking service.
- **Real-Life Example:** A card swipe machine shouldn't need to be rebuilt completely when a new digital wallet is introduced.
- **Why it Fails:** Every time you want to add a new payment method (like NetBanking), you have to change and re-test the old code, which might break it.

- **What TO Do Instead:** Create a base class `Payment` . Make separate child classes for `UpiPayment` , `CardPayment` , and `CashPayment` .

The Rule

Use separate classes for different types instead of long if-else chains.

Mistake 4: Putting everything into one massive class

- **What NOT To Do:** Do not put movies, screens, seat layouts, payments, and menus into one giant file.
- **Real-Life Example:** One person cannot be the ticket seller, projector operator, security guard, and cleaner all at once. If they get sick, the cinema stops.
- **Why it Fails:** It creates messy code that is very hard to read, test, or fix.
- **What TO Do Instead:** Split the system into small, focused classes (One class per file). `Movie` for data, `PriceCalculator` for math, `TicketPrinter` for printing.

The Rule

Every class should have exactly one clear job.

Mistake 5: Making "Seat Layout" or "Price" separate classes

- **What NOT To Do:** Do not create a class for `SeatLayout` or `Price` .
- **Real-Life Example:** A price is just a number. A seat layout is just a visual drawing of the seats on a screen.
- **Why it Fails:** Creating extra empty classes adds unnecessary confusion without providing any benefit.
- **What TO Do Instead:** Keep `Price` as a simple number (`double`). Keep `Seat Layout` as a display method.

The Rule

Do not make classes for simple numbers or basic display views.

4. Final Clean Class List



- Movie
- Seat
- Screen
- Cinema
- Show
- ShowSeat
- Customer
- Booking

- PriceCalculator
- TicketPrinter
- BookingService

Payment Classes

- Payment (Abstract)
- UpiPayment
- CardPayment
- CashPayment